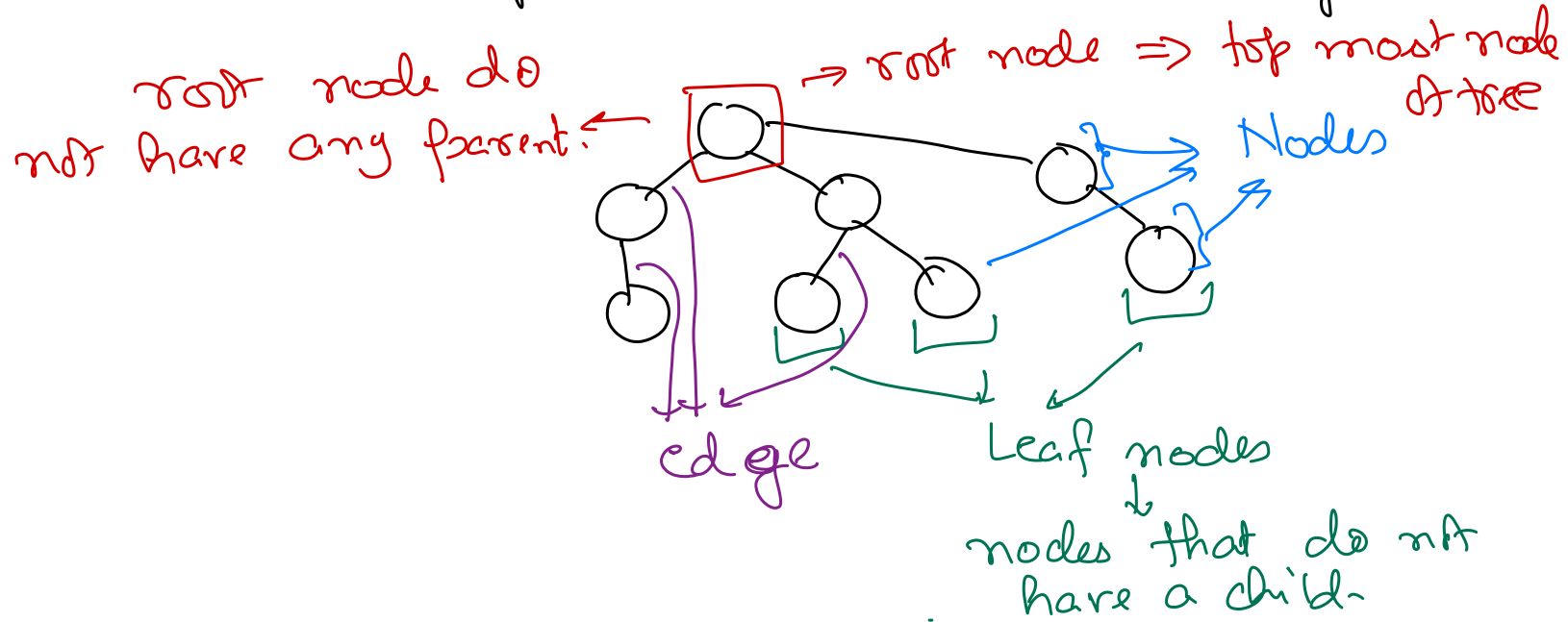
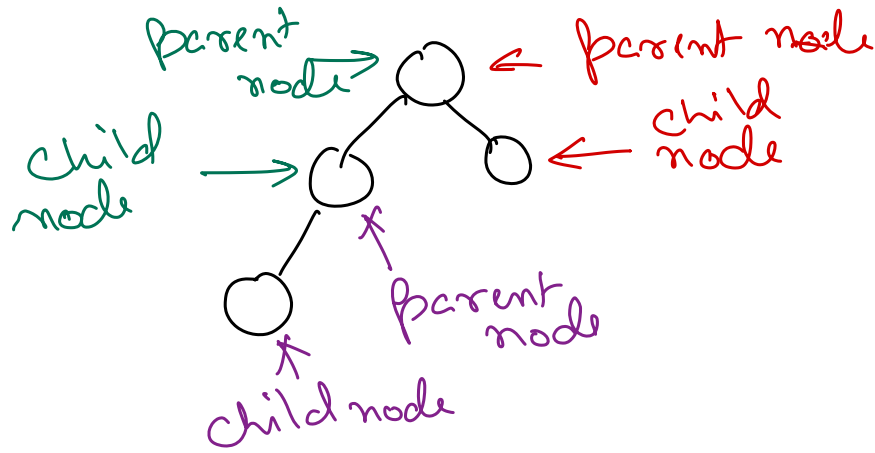


Tree

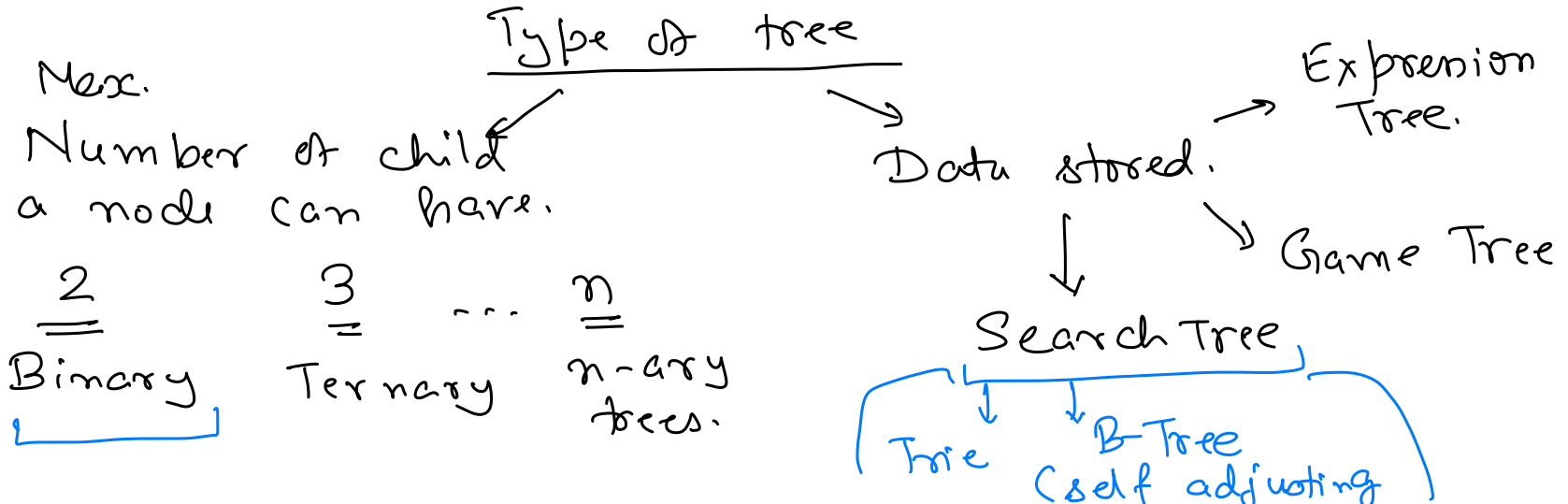
- Non-linear data structure.
- Tree is a collection of nodes.
- Tree stores parent-child relationship.

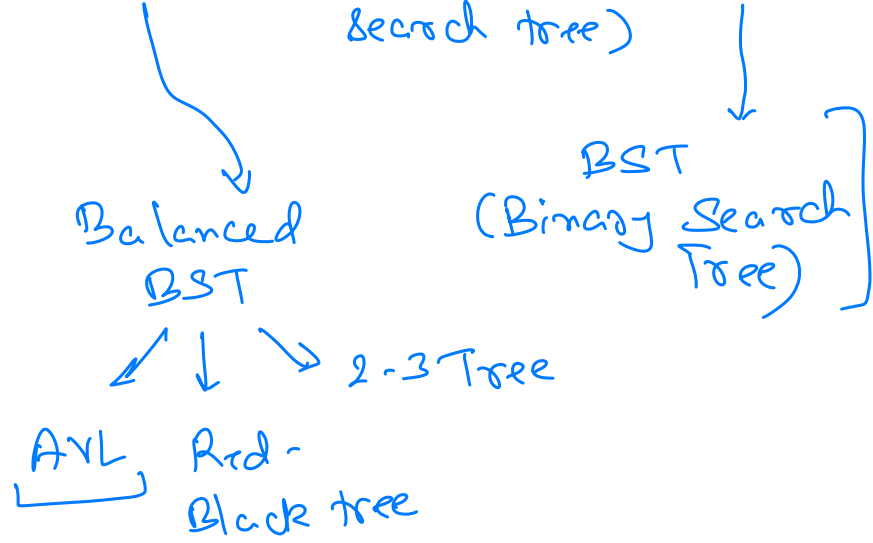




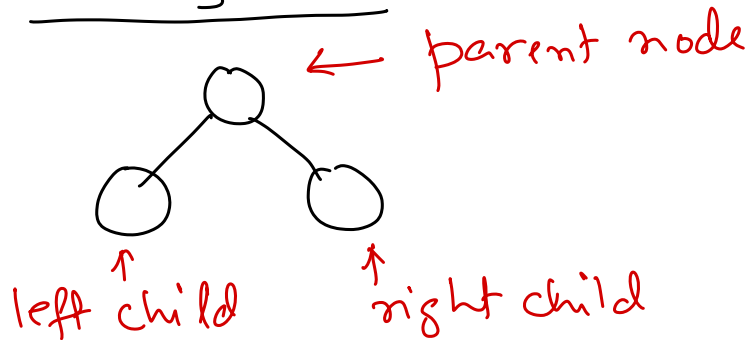
Each node keeps track of its child nodes.

child node do not keep track of parent node.





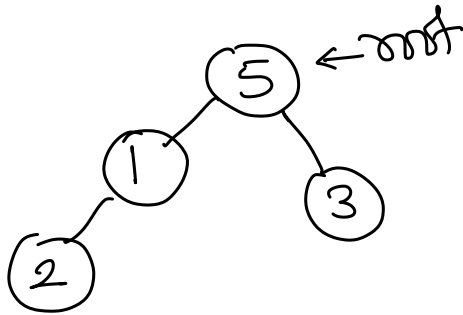
Binary Tree



Binary Tree Traversal

Depth first

① ③ ② → In order
③ ① ② → Pre order
① ② ③ → Post order



Breadth first

↳ Level order

traversal (root) =

root → empty

done, if root is empty.

① traverse (root's left child)

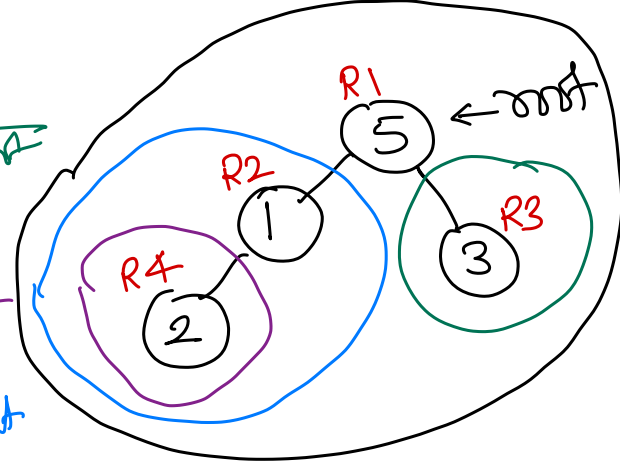
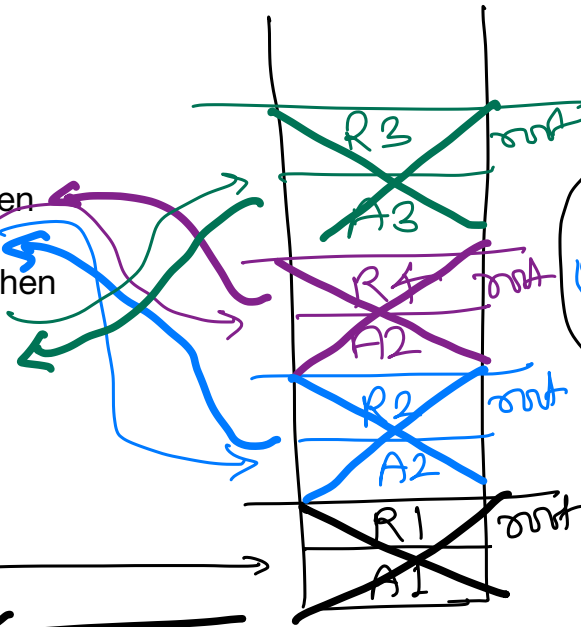
② traverse (root's right child)

③ process root

otherwise.

PreOrder(root)

- if (root is empty) then
 - Stop.
- Process root node's data.
- If (root node's left child exists) then
 - PreOrder(root's left child). **A2**
- If (root node's right child exists) then
 - PreOrder(root's right child) **A3**
- Stop.



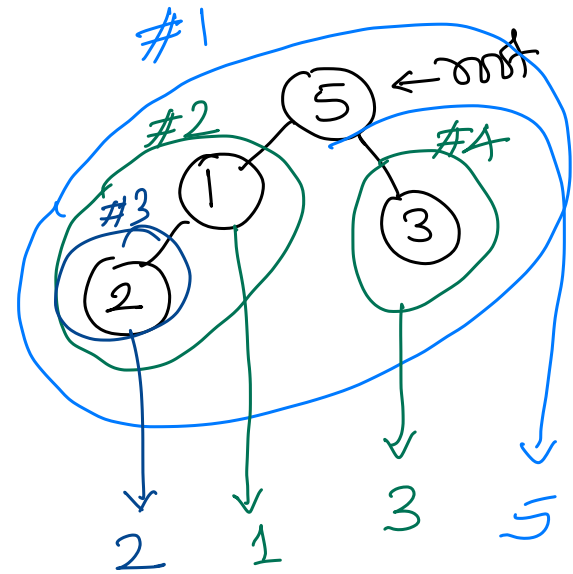
Pre Order (root) **A1**

O/P: 5 1 2 3

PostOrder(root)

- if (root is empty) then
 - Stop.
- If (root node's left child exists) then
 - PostOrder(root's left child).
- If (root node's right child exists) then
 - PostOrder(root's right child).
- Process root node's data.
- Stop.

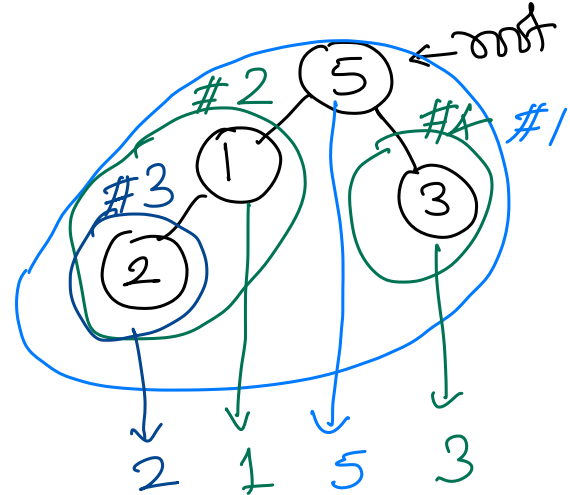
o/p:
2 1 3 5

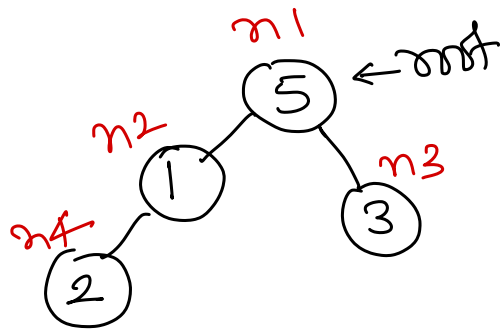


InOrder(root)

- if (root is empty) then
 - Stop.
- If (root node's left child exists) then
 - InOrder(root's left child).
- Process root node's data.
- If (root node's right child exists) then
 - InOrder(root's right child).
- Stop.

o/p:
2 1 5 3

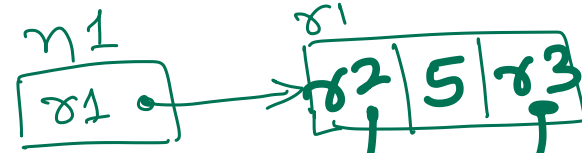




```

class BTreeNode {
    int data;
    BTreeNode leftChild;
    BTreeNode rightChild;
}
  
```

BTreeNode n1 = new BTreeNode()



BTreeNode n2 = new BTreeNode()



BTreeNode n3 = new BTreeNode()



BTreeNode n4 = new BTreeNode()



n1. data = 5;

n1. leftChild = n2;

n1. rightChild = n3;

n3. data = 3;

n3. leftChild = null;

n3. rightChild = null;

n2. data = 1;

n2. leftChild = n4;

n2. rightChild = null;

n4. data = 2;

n4. leftChild = null;

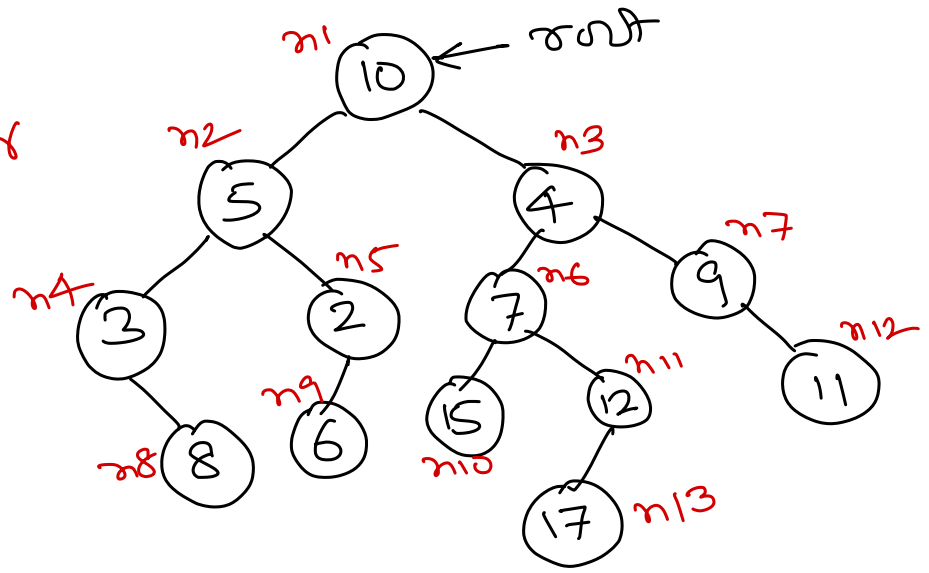
n4. rightChild = null;

Exercise

→ Find o/p of Inorder
Preorder &
Postorder Traversal.

→ Hard code the
tree.

→ Find o/p of all traversals.



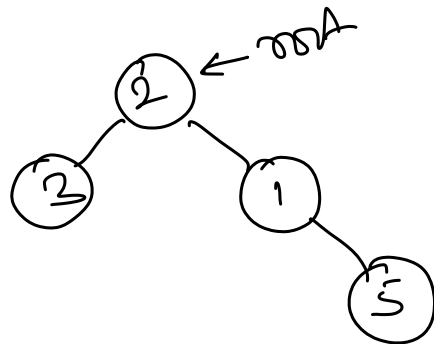
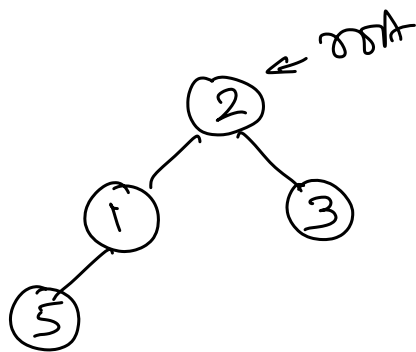
① Count number of nodes in a binary tree.

$$\text{Count Nodes (root)} = \begin{cases} 0, & \text{if root is empty,} \\ 1 + \text{Count Nodes (root.left child)} \\ & + \text{Count Nodes (root.right child)} \\ \text{Otherwise} \end{cases}$$

② Count number of leaf nodes in a binary tree.

③ Count frequency of occurrence of a value in a binary tree.

④ Invert a binary tree. / Find mirror image of a binary tree.

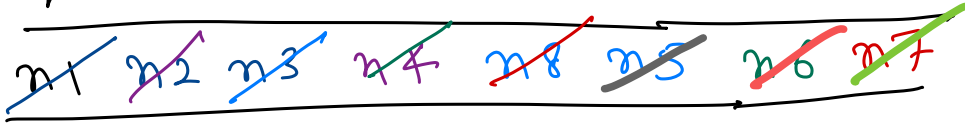


⑤ Print contents of a binary tree with value of nodes on same level on one line.

5
2 3
↓ 1 4 6
7 9

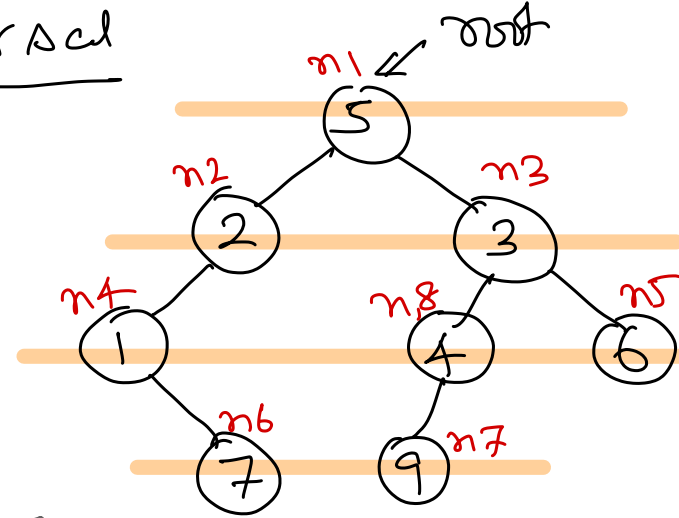
Level order Traversal

queue



o/p: 5 2 3 1 4 6 7 9

Current → ~~n1~~ ~~n2~~ ~~n3~~ ~~n4~~ ~~n5~~ ~~n6~~ n7



LevelOrderTraversal(root).

- if (root is empty) then
 - Stop.
- Add the root node to the queue.
- while (queue is not empty) do
 - Get a node from queue.
 - Process the node.
 - Add the non-empty childs of the node to the queue.
- Stop.

BST Property

Value of nodes in
left subtree $<$ value of
parent $<$

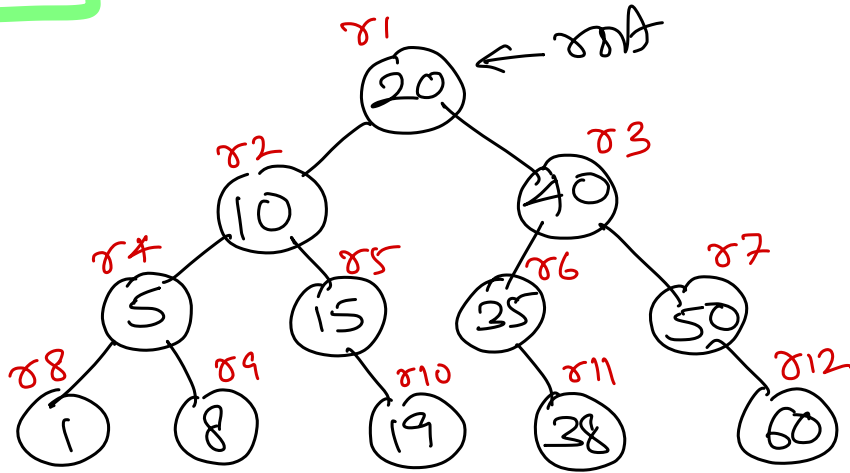
value of nodes in right
subtree.

Binary Search Tree

→ A binary tree in which each node satisfies Binary Search Tree (BST) property.

Search(root, element)

$\left. \begin{array}{l} \text{root} \rightarrow r1 \\ \text{element} \rightarrow 5 \end{array} \right\} \text{search}(r1, 5)$
↓



Search(σ_2 , 5)

$\Downarrow$

Search(σ_4 , 5) ✓

Search(σ_1 , 9)

$\Downarrow$

Search(σ_2 , 9)

$\Downarrow$

Search(σ_4 , 9)

$\Downarrow$

Search(σ_9 , 9)

$\Downarrow$

Search(σ_{null} , 9)
✗

Search (root, element)

→ if (root == empty) then

→ return false

→ if (root's data == element) then

→ return true

→ if (element < root's data) then

→ return Search (root's left child, element)

→ return Search (root's right child, element)

Tail Recursion